

Học Sâu (Deep Learning)

Chương 5: Nền tảng của Học máy (Fundamentals of ML)

Đại học Đông Á

August 14, 2026

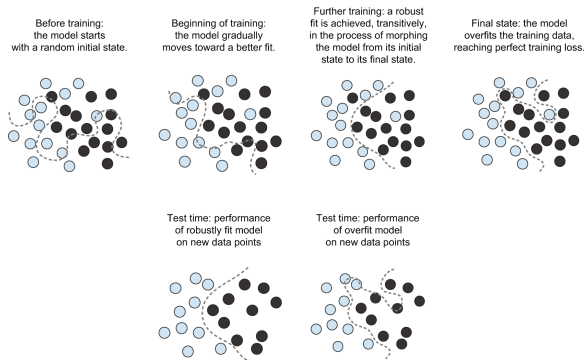
Nội dung chương

1. Tối ưu hóa (Optimization) và Khái quát hóa (Generalization)
2. Tác động của Dữ liệu Nhiễu (Noisy Data)
3. Các phương pháp Đánh giá Mô hình
4. Dung lượng Mô hình (Model Capacity)
5. Các Phương pháp Regularization
6. Giả thuyết Đa tạp (The Manifold Hypothesis)
7. Tổng kết

- **Tối ưu hóa (Optimization):** Quá trình điều chỉnh mô hình để đạt hiệu suất tốt nhất trên tập dữ liệu huấn luyện (*Training data*).
- **Khái quát hóa (Generalization):** Khả năng mô hình dự đoán chính xác trên dữ liệu hoàn toàn mới chưa từng thấy (*Test data*).
- **Mục tiêu:** Đạt được khả năng Generalization cao nhất. Tuy nhiên, thuật toán Gradient Descent chỉ có thể trực tiếp tác động vào quá trình Optimization.
- **Mâu thuẫn:** Sau một thời gian huấn luyện nhất định, Optimization càng tốt thì Generalization lại càng đi xuống.

Hiện tượng Underfitting và Overfitting

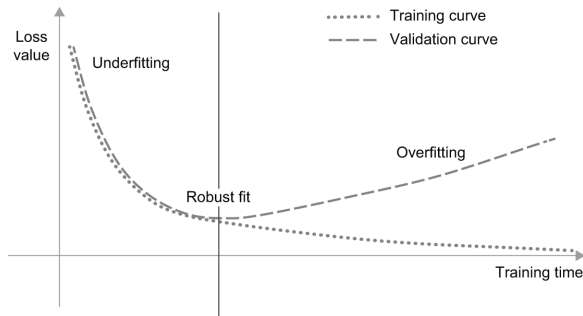
- **Underfitting (Chưa khớp):** Mô hình quá đơn giản, không học được quy luật trong dữ liệu. (Ví dụ: Dùng đường thẳng tuyến tính để khớp dữ liệu phân bố parabol).
- **Robust Fit (Khớp chuẩn):** Mô hình học được xu hướng thực sự của dữ liệu.
- **Overfitting (Quá khớp):** Mô hình quá phức tạp, học thuộc lòng cả các "nhiều" (noise) của dữ liệu huấn luyện, khiến đường biên quyết định cong queo, mất đi tính tổng quát.



Hình 5.1: Mô phỏng Underfitting, Robust Fit và Overfitting

Vòng đời điển hình của mô hình (Universal Overfitting)

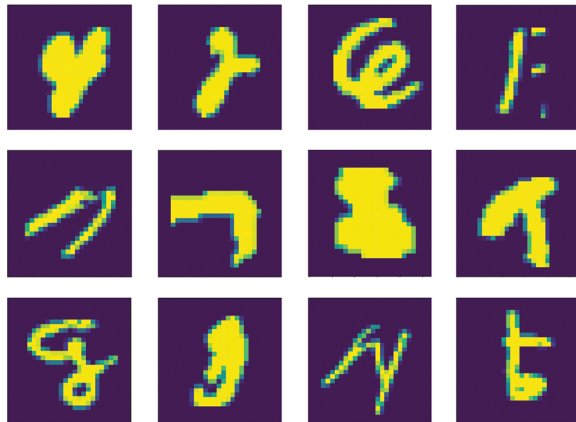
- Ở giai đoạn đầu, Loss trên tập Train và tập Validation cùng giảm, các tham số đang liên tục điều chỉnh đúng hướng → Giai đoạn Underfitting.
- Sau một số epoch nhất định, Validation Loss bắt đầu chứng lại và tăng vọt trở lại, dù Training Loss vẫn tiếp tục giảm tiệm cận 0 → **Giai đoạn Overfitting**.



Hình 5.2: Hành vi Overfitting kinh điển

Dữ liệu không hợp lệ (Invalid Inputs)

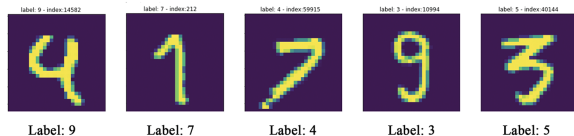
- Trong thực tế, dữ liệu thường không hoàn hảo, chứa nhiều tạp âm, nhòe nhoẹt.
- Ví dụ: Tập dữ liệu ảnh số viết tay MNIST có chứa những hình ảnh bị lỗi, mờ đến mức mắt người cũng không thể nhận diện được.
- Nếu mạng nơ-ron có đủ dung lượng, nó sẽ cố gắng "học vẹt" những hình nhiễu này bằng cách dùng một phần tài nguyên mạng ghi nhớ chúng. Điều này làm suy giảm tính khái quát.



Hình 5.3: Các ảnh bị nhiễu nặng trong tập MNIST

Dữ liệu bị gán nhãn sai (Mislabeled Data)

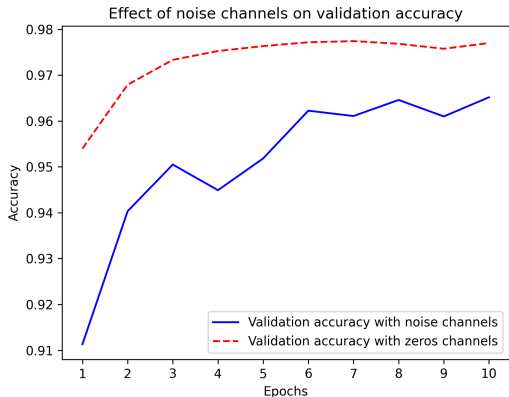
- Do sai sót chủ quan của con người (Human Error), một số dữ liệu có thể bị gán nhãn (label) hoàn toàn sai.
- Ví dụ: Rõ ràng là số 4 nhưng lại gán nhãn là số 9 trong tập MNIST.
- Nếu mô hình cố gắng uốn cong ranh giới quyết định (decision boundary) để "chữa" lại những nhãn sai này, nó sẽ nhận diện sai hàng loạt các hình số 4 tương tự trong tương lai.



Hình 5.4: Dữ liệu gán nhãn sai trong MNIST

Sự phình to của Kênh Nhiễu (Noise Channels)

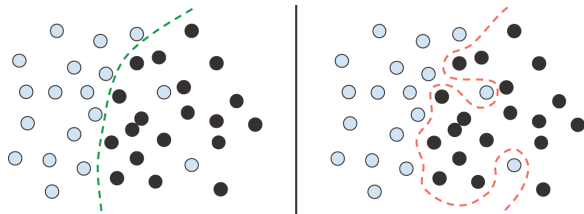
- Thí nghiệm: Thêm một số ma trận toàn nhiễu trắng (white noise) hoặc toàn số 0 nối thêm vào dữ liệu MNIST ban đầu.
- Kết quả: Kể cả khi dữ liệu MNIST hợp lệ vẫn nguyên vẹn, mạng bị phân tâm bởi kênh nhiễu.
- Validation Accuracy của bộ dữ liệu chứa Noise Channels giảm rõ rệt. Lý do: Mô hình vô tình tìm ra các mẫu (patterns) ngẫu nhiên giả mạo trong tập nhiễu và sử dụng chúng để ra quyết định → Suy giảm Generalization.



Hình 5.5: Tác động của kênh nhiễu đến Accuracy

Ngoại lai (Outliers) và Sự quá khớp

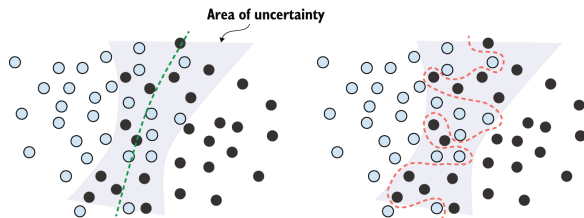
- Các điểm nhiễu thường đóng vai trò là "Ngoại lai" (Outliers), tức là các điểm dữ liệu nằm lạc lõng giữa quần thể lớp khác.
- Việc Gradient Descent nỗ lực cực độ để phân loại đúng 100% tập huấn luyện sẽ ép đường ranh giới quyết định vươn xa ra ngoài để bọc lấy outlier.
- Hậu quả: Ranh giới quyết định trở nên góc cạnh thay vì là một đường cong mượt mà, bao quát.



Hình 5.6: Robust fit (trái) vs. Overfitting ngoại lai (phải)

Đặc trưng mơ hồ (Ambiguous features)

- Đôi khi ranh giới giữa hai nhóm dữ liệu có vùng giao thoa. Tại vùng này, một dữ liệu có xác suất 50% thuộc lớp A và 50% thuộc lớp B (Uncertainty).
- Thay vì vạch một đường thẳng cắt đôi đám giao thoa mang tính chất thống kê này, Overfitting sẽ biến đường biên thành hình răng cưa uốn éo cố phân chia tách bạch.
- Khắc phục: Phải cho phép mô hình phạm lỗi trên tập Train (Training Loss không bao giờ bằng 0).



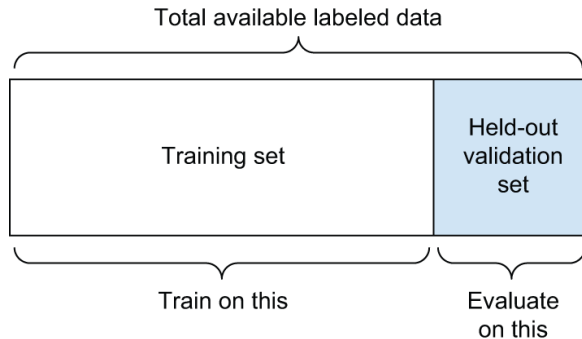
Hình 5.7: Overfitting trên vùng dữ liệu bất định

Chia 3 tập: Train, Validation và Test

- Tại sao không chỉ có 2 tập Train và Test?
- Quá trình huấn luyện không chỉ là điều chỉnh "Tham số" (Weights/Biases) qua Gradient Descent mà còn là điều chỉnh "Siêu tham số" (Hyperparameters - số lớp ẩn, số nơ-ron, learning rate...).
- Chúng ta dò tìm các cấu hình siêu tham số này để cải thiện điểm số trên tập Validation.
- **Information Leak:** Việc liên tục dùng tập Validation làm la bàn để tối ưu siêu tham số sẽ khiến mô hình gián tiếp "học lỏm" cấu trúc tập Validation → Overfit trên chính Validation.
- Do đó, ta bắt buộc phải có tập thứ 3: **Tập Test** (Hoàn toàn mới, chỉ chạy đúng 1 lần trước khi đưa ứng dụng vào thực tế).

Tách tập Validation đơn giản (Hold-out Validation)

- Xáo trộn dữ liệu (Shuffle) và tách một phần cố định (vd: 20%) từ tập Training để làm tập Validation.
- Phương pháp này rất nhanh gọn, thường dùng khi khối lượng dữ liệu khổng lồ (hàng trăm ngàn điểm dữ liệu).
- Nhược điểm: Nếu dữ liệu ít, tập Validation sẽ không mang tính đại diện thống kê cho toàn bộ phân phối, dẫn tới sai số đánh giá lớn khi đổi seed chia tách.



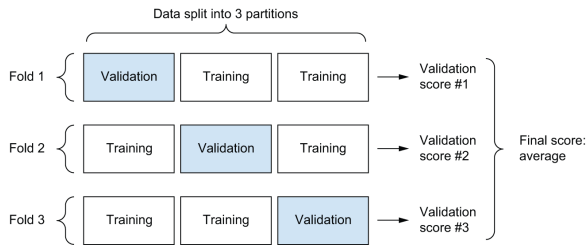
Hình 5.8: Quy trình Hold-out Validation

Mã nguồn: Hold-out Validation

```
1 import numpy as np
2
3 num_val_samples = 10000
4 np.random.shuffle(data) # Xao tron ngau nhien
5
6 # Tach 10k mau lam Validation
7 val_data = data[:num_val_samples]
8 train_data = data[num_val_samples:]
9
10 model = get_model()
11 model.fit(train_data, epochs=20)
12 val_score = model.evaluate(val_data)
13
14 # Sau khi dieu chinh hyperparameter thanh cong,
15 # ta huan luyen lai 1 mo hinh tren toan bo Train + Validation
16 model = get_model()
17 model.fit(np.concatenate([train_data, val_data]), epochs=20)
18 test_score = model.evaluate(test_data)
```

Đánh giá chéo K-Lần (K-Fold Validation)

- Khi dữ liệu cực kỳ thưa thớt, việc Hold-out 1/5 lượng dữ liệu sẽ làm lãng phí cơ hội học tập của mạng.
- **K-Fold Validation:** Chia dữ liệu thành K phần (Fold). Lặp lại K vòng huấn luyện khác nhau. Ở vòng i , lấy Fold i làm Validation, gộp tất cả $K - 1$ Fold còn lại làm Training.
- Điểm số sau cùng là giá trị trung bình của K điểm số đánh giá.
- Độ chính xác cao, nhưng chi phí tài nguyên gấp K lần.



Hình 5.9: Quy trình K-Fold Validation (với $K=3$)

Mã nguồn: K-Fold Validation

```
1 k = 4
2 num_val_samples = len(data) // k
3 val_scores = []
4
5 for fold in range(k):
6     # Lay ra phan Validation riêng
7     val_data = data[num_val_samples * fold : num_val_samples * (fold + 1)]
8
9     # Gop phan dau va cuoi de tao thanh tap Train
10    train_data = np.concatenate(
11        [data[: num_val_samples * fold], data[num_val_samples * (fold + 1) :]], axis
12        =0
13    )
14
15    model = get_model() # Khoi tao lai mo hinh moi hoan toan!
16    model.fit(train_data, epochs=100, batch_size=16)
17    score = model.evaluate(val_data)
18    val_scores.append(score)
19
20 average_val_score = np.mean(val_scores) # Lay trung binh
```

Xử lý khi Validation Loss không chịu giảm

- Đôi khi trong quá trình huấn luyện, Loss hoàn toàn không giảm và độ chính xác giữ nguyên ở ngưỡng Random Guess (chọn bừa).
- Điều này thường liên quan đến **Learning Rate** (Tốc độ học) đang quá cao hoặc quá thấp.
- Ví dụ: Tốc độ học khổng lồ '1.0' sẽ khiến các bước Gradient bay nhảy hỗn loạn trong không gian hàm mất mát.

```
1 # Tình huống khiến mô hình "chết lâm sàng", Accuracy không thể tăng
2 model.compile(
3     optimizer=keras.optimizers.RMSprop(learning_rate=1.0),
4     loss="sparse_categorical_crossentropy",
5     metrics=["accuracy"],
6 )
```


Điều chỉnh Learning Rate và Batch Size

- Khi hạ Learning Rate xuống mức tiêu chuẩn (ví dụ '1e-2', '1e-3'), Loss sẽ bắt đầu giảm trở lại.
- Nếu mạng vẫn không học, thử Tăng Batch Size. Lô dữ liệu lớn hơn sẽ tạo ra các vector đạo hàm ít nhiễu hơn, cung cấp định hướng chính xác hơn.

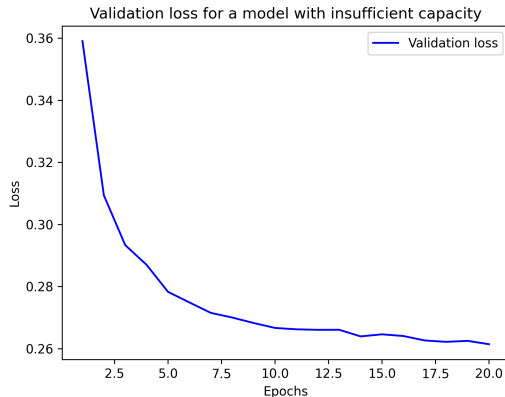
```
1 # Giảm tốc độ học giúp mô hình hội sinh
2 model.compile(
3     optimizer=keras.optimizers.RMSprop(learning_rate=1e-2),
4     loss="sparse_categorical_crossentropy",
5     metrics=["accuracy"],
6 )
7 model.fit(train_images, train_labels, epochs=10, batch_size=128)
```

Dung lượng Mô hình (Model Capacity) là gì?

- Dung lượng của mạng nơ-ron tỷ lệ thuận với số lượng tham số (trọng số) mà nó sở hữu. Số lượng layer và số unit quyết định đại lượng này.
- **Dung lượng quá nhỏ:** Không đủ khả năng "ghi nhớ" các quy luật biểu diễn phi tuyến tính phức tạp → Underfitting.
- **Dung lượng quá lớn:** Quá dư thừa, mạng dễ dàng học vẹt (như một cuốn từ điển mapping từng input ra output) → Overfitting cực nhanh.
- Mục tiêu của người học sâu là thiết kế mạng có dung lượng vừa khít, có khả năng giảm loss tối đa nhưng trì hoãn việc overfit muộn nhất.

Khi Dung lượng Mô hình Không Đủ (Insufficient)

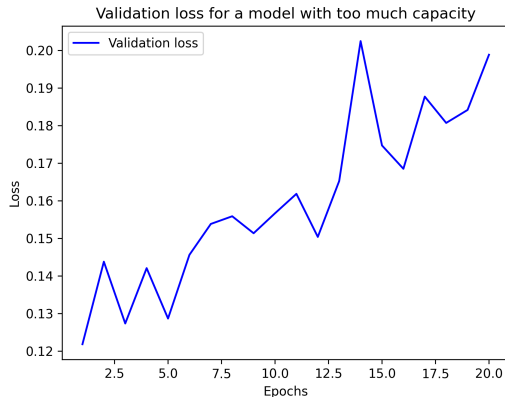
- Thử nghiệm dùng mạng hồi quy Logistic quá nhỏ (ví dụ cấu trúc nông chỉ xuất thẳng ra softmax) trên MNIST.
- Kết quả: Validation Loss không bao giờ xuống tới mức thấp mong muốn. Loss cứ lặp lại ở mức 0.26 và giữ nguyên (Stalled).
- Hiện tượng này xác nhận mạng không đủ sức mạnh để khái quát không gian giả thuyết phức tạp của hình ảnh.



Hình 5.10: Loss khi Model Capacity quá yếu

Khi Dung lượng Mô hình Quá Lớn (Excessive)

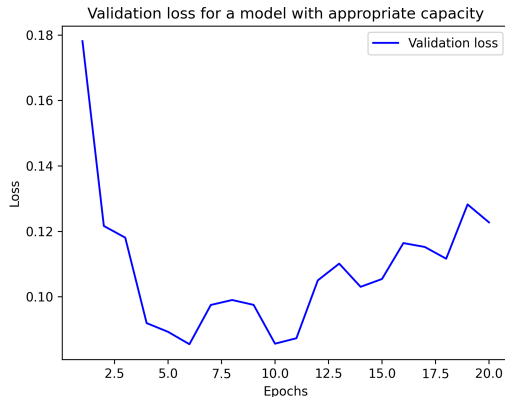
- Thử nghiệm dùng mạng khổng lồ (3 lớp ẩn, mỗi lớp 2048 nơ-ron - hàng chục triệu tham số).
- Kết quả: Do sức mạnh học vẹt quá kinh khủng, nó nhớ toàn bộ tập Training trong vài giây. Validation Loss chạm đáy ngay tại Epoch 1, sau đó vọt lên mạnh mẽ.
- Đường Validation Loss cũng rất nhiều (noise) chứng tỏ sự mất ổn định.



Hình 5.11: Loss khi Model Capacity quá lớn

Dung lượng Mô hình Lý Tưởng (Correct Capacity)

- Mô hình có 2 lớp ẩn (mỗi lớp 128 units).
- Validation Loss giảm từ từ và duy trì ở mức tối thiểu trước khi bắt đầu có dấu hiệu tăng nhẹ ở epoch thứ 8.
- Đây là thiết kế kiến trúc (Architecture) cân bằng tuyệt vời giữa tối ưu hóa và khái quát hóa dữ liệu hình chữ số MNIST.



Hình 5.12: Loss khi Model Capacity phù hợp

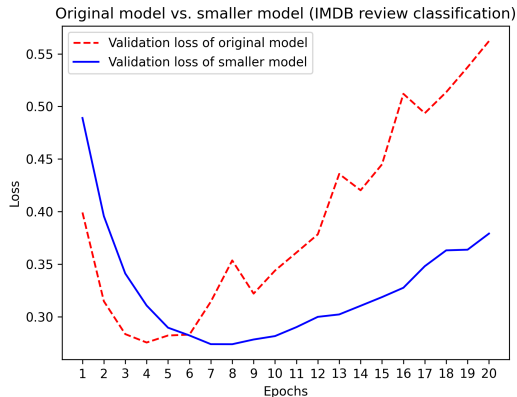
So sánh trên bài toán IMDB (Phân tích cảm xúc)

- Mạng phân tích phim gốc ở Chương 4 sử dụng 2 lớp, 16 units.
- Hãy thử nghiệm giảm xuống còn 4 units, và tăng vọt lên 512 units.

```
1 # Mạng IMDB gốc (16 units)
2 model_base = keras.Sequential([
3     layers.Dense(16, activation="relu"),
4     layers.Dense(16, activation="relu"),
5     layers.Dense(1, activation="sigmoid")
6 ])
7
8 # Mạng IMDB nhỏ (4 units) -> Mục tiêu giảm thiểu Overfitting
9 model_small = keras.Sequential([
10     layers.Dense(4, activation="relu"),
11     layers.Dense(4, activation="relu"),
12     layers.Dense(1, activation="sigmoid")
13 ])
```

Trực quan: Mạng Gốc (16 units) vs Mạng Nhỏ Hơn (4 units)

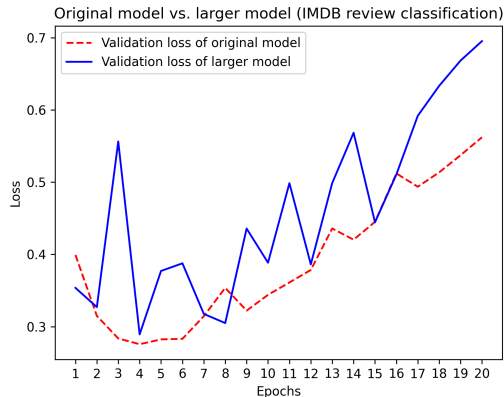
- Đường có dấu chấm tròn (Dots) đại diện cho Mạng gốc 16 units.
- Đường đứt khúc (Solid/Crosses) đại diện cho Mạng nhỏ 4 units.
- Phân tích: Mạng nhỏ (Dung lượng thấp) phải mất nhiều epoch hơn (6 epoch) để bắt đầu có dấu hiệu tăng Validation Loss. Khi Overfit, đồ thị cũng dốc thoải chậm hơn rất nhiều so với bản gốc.
- Kết luận: Giảm dung lượng mạng là kỹ thuật Regularization gốc rễ.



Hình 5.13: Mạng nhỏ (Crosses) vs Mạng gốc (Dots)

Trực quan: Mạng Gốc (16 units) vs Mạng Lớn Hơn (512 units)

- Đường có dấu chấm tròn (Dots) đại diện cho Mạng gốc 16 units.
- Đường đứt khúc đại diện cho Mạng 512 units.
- Phân tích: Mạng 512 units rơi vào tình trạng "Ghi nhớ thần tốc". Ngay từ epoch đầu tiên nó đã Overfit dữ liệu và vọt lên rất cao, biên độ của Loss bị nhiễu lớn.



Hình 5.14: Mạng lớn (Crosses) vs Mạng gốc (Dots)

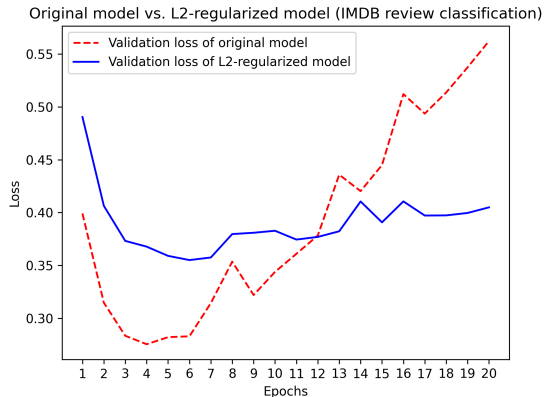
Weight Regularization (Phạt Trọng Số L1 / L2)

- Áp dụng nguyên lý Ockham's razor: Lời giải thích đơn giản hơn thường có xu hướng đúng. Một mô hình có các trọng số siêu nhỏ phân bố đều thì đơn giản hơn một mô hình chứa các trọng số khổng lồ bất thường.
- **L1 Regularization:** Thêm mức phạt tỷ lệ với trị tuyệt đối của trọng số $|W|$. Sẽ ép nhiều trọng số về đúng bằng 0 (Tạo sự thưa thớt - Sparsity).
- **L2 Regularization (Weight Decay):** Thêm mức phạt tỷ lệ thuận với bình phương trọng số W^2 . Ép trọng số nhỏ về tiệm cận 0.

```
1 from keras.regularizers import l2, l1_l2
2
3 # Thêm tham số kernel_regularizer để tích hợp Phạt trọng số
4 model = keras.Sequential([
5     layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
6     layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
7     layers.Dense(1, activation="sigmoid")
8 ])
```

Hiệu quả của Phạt trọng số L2

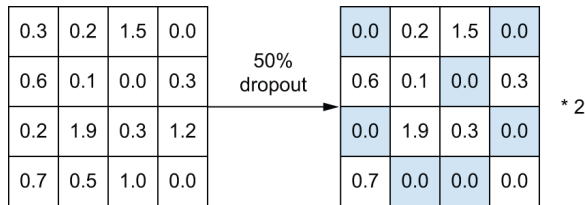
- Tham số ' $\lambda(0.002)$ ' sẽ cộng thêm $0.002 \times W^2$ vào hàm Loss gốc. Lưu ý, khoản phạt này chỉ tác động lúc Huấn luyện, tắt hoàn toàn lúc Kiểm tra.
- Biểu đồ cho thấy mạng dùng L2 (Crosses) duy trì Validation Loss ở mức an toàn và lâu overfit hơn đáng kể so với mạng không phạt.
- Thường được dùng tốt nhất trên các mô hình có kích cỡ bé và vừa.



Hình 5.15: Hiệu quả của L2 Regularization (Crosses)

Kỹ thuật Dropout (Ngắt Nơ-ron Ngẫu nhiên)

- Kỹ thuật cách mạng do **Geoffrey Hinton** phát minh.
- Cơ chế: Trong từng bước tối ưu, tại một layer, đột ngột vô hiệu hóa ngẫu nhiên (ép về giá trị 0) một tỷ lệ (rate) các nơ-ron, thường là $0.2 \sim 0.5$.
- Ngăn chặn hiện tượng "đồng thích nghi" (Co-adaptation): Các nơ-ron không thể ỷ lại, dựa dẫm học chung một tính năng với nơ-ron khác → Buộc phải tự hình thành những đặc trưng độc lập và mạnh mẽ.



Hình 5.16: Quá trình Dropout tại thời điểm Training

Cơ chế cân bằng Toán học của Dropout

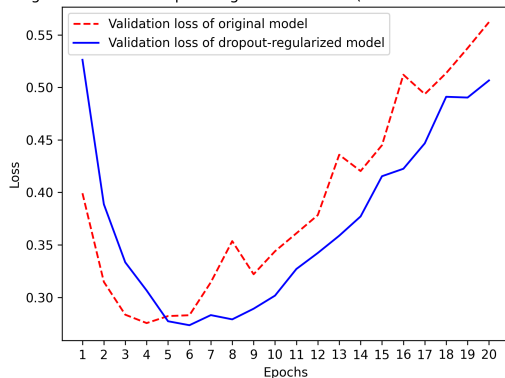
- **Khi Training:** Ngắt 50% nơ-ron ($\text{rate} = 0.5$).
- **Khi Testing (Inference):** Tuyệt đối không Dropout! Mô hình chạy full công suất.
- Vấn đề: Nếu lúc học có 50 người kéo co, lúc thi có tận 100 người kéo co thì tổng lực (tổng Activation) bị tăng gấp đôi, làm vỡ hàm Loss.
- Giải pháp Keras: Tại lúc học, các kích hoạt còn sống sót sẽ được **Scale Up** (nhân cho $\frac{1}{1-\text{rate}} = 2$).

```
1 model = keras.Sequential([
2     layers.Dense(16, activation="relu"),
3     layers.Dropout(0.5), # Nhét lớp Dropout ngay sau lớp Dense
4     layers.Dense(16, activation="relu"),
5     layers.Dropout(0.5),
6     layers.Dense(1, activation="sigmoid")
7 ])
```

Hiệu quả của Dropout

- Trong biểu đồ này, mạng sử dụng Dropout rate = 0.5 (Crosses) đã kháng cự lại sự Overfit vô cùng kiên cường.
- Đây chính là kỹ thuật Regularization hiệu quả và được áp dụng gần như mặc định cho các mô hình Deep Learning cực sâu hiện tại.

Original model vs. dropout-regularized model (IMDB review classification)



Hình 5.17: Hiệu quả của Dropout (Crosses)

Kỹ thuật tính năng (Feature Engineering)

- Feature Engineering là quá trình sử dụng kiến thức, quy luật chuyên gia (Domain Knowledge) để biến đổi dữ liệu đầu vào sao cho nó trở nên thân thiện nhất với thuật toán ML/DL.
- DL hiện đại có khả năng trích xuất tính năng ẩn một cách tự động, tuy nhiên với các tập dữ liệu nhỏ hoặc bài toán đặc thù, nếu ta thiết kế Feature Engineering tốt, mô hình sẽ tốn ít tài nguyên hơn, chạy nhanh hơn và chính xác hơn.
- Một bài toán kinh điển: Nhận diện thời gian trên đồng hồ kim.

Ví dụ: Bài toán đọc Đồng hồ

- **Cách 1 (Không tinh tế):** Dùng dữ liệu pixel ảnh trực tiếp, nuôi bằng mạng chập CNN. Cực kỳ lãng phí tài nguyên và dữ liệu huấn luyện khổng lồ.
- **Cách 2 (Trung gian):** Code Python trích xuất tọa độ (x, y) của đầu kim đồng hồ. Cho thuật toán ML dễ dàng học các số tọa độ.
- **Cách 3 (Feature đỉnh cao):** Chuyển (x, y) sang tọa độ cực: góc độ θ . Lúc này, bài toán giải bằng 1 hàm làm tròn chia chẵn lẻ mà không cần AI!

Raw data: pixel grid		
Better features: clock hands' coordinates	$\{x1: 0.7,$ $y1: 0.7\}$ $\{x2: 0.5,$ $y2: 0.0\}$	$\{x1: 0.0,$ $y1: 1.0\}$ $\{x2: -0.38,$ $y2: 0.32\}$
Even better features: angles of clock hands	theta1: 45 theta2: 0	theta1: 90 theta2: 140

Hình 5.18: Feature Engineering cho bài toán đọc thời gian

Giả thuyết Đa tạp (The Manifold Hypothesis) là gì?

- Một ảnh màu 256×256 có 3 kênh màu RGB tạo ra một không gian lên tới hàng tỷ tỷ chiều (cao hơn vô hạn so với số nguyên tử trong vũ trụ).
- Nếu lấy ngẫu nhiên 1 tập pixel bất kỳ, xác suất tạo ra hình dạng "hợp lý" là 0.
- **Manifold Hypothesis:** Tuy nhiên, các dữ liệu có ý nghĩa thực tế của loài người (ảnh tự nhiên, khuôn mặt, chữ số MNIST, âm thanh) chỉ nằm tập trung tại một dải không gian con cấu trúc cao, uốn lượn gọn gàng, gọi là **Đa tạp (Manifold)**.
- Học sâu chính là quá trình vuốt phẳng dần (uncrumpling) các đa tạp rối rắm ẩn sâu này.

Nội suy Tuyến tính vs Nội suy Đa tạp

- Khi mô hình dự đoán một điểm chưa từng thấy, nó thực hiện **Nội suy (Interpolation)**.
- Nội suy tuyến tính (đường thẳng nối giữa 2 chữ số) thường tạo ra những giá trị văng ra ngoài không gian của chữ số $\rightarrow$ Tức là tạo ra khối hình vô nghĩa (Ảnh trái).
- Học sâu bám chặt vào đường cong Đa tạp, nội suy dọc theo mặt phẳng cong (Manifold Interpolation), mang lại hình chuyển tiếp siêu mượt (Ảnh phải).



Manifold interpolation
(intermediate point on
the latent manifold)

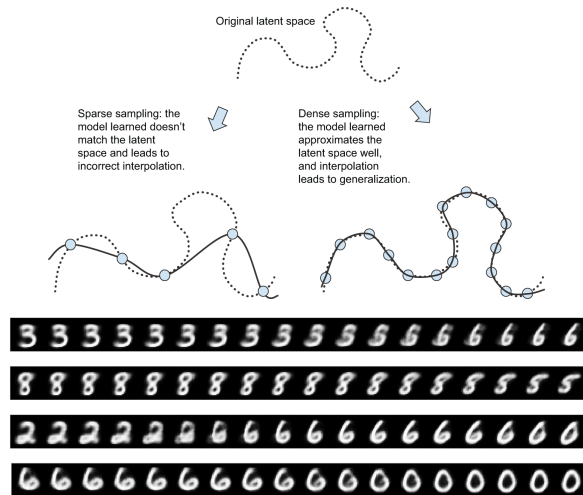


Linear interpolation
(average in the
encoding space)

Hình 5.19: Linear Interpolation (sai) vs Manifold (đúng)

Mật độ Lấy mẫu (Dense Sampling)

- Vì học sâu bản chất chỉ là "khớp đường cong", nên để học được trọn vẹn Đa tạp cong uốn lượn, chúng ta cần khối lượng dữ liệu khổng lồ lấp đầy bề mặt (Dense Sampling).
- Không gian ẩn (Latent space) của mạng học được sẽ tự động gom cụm các chữ số theo từng đảo (cluster) giống nhau (Ảnh dưới).
- "Thuật toán tối tân không bù đắp được dữ liệu thừa thớt."



Hình 5.20: Dense Sampling & MNIST Manifold

- Cốt lõi của Học máy là duy trì ranh giới mỏng manh giữa việc cực đại hóa **Optimization** trên tập Train mà không phá hủy **Generalization** trên tập Test.
- **Overfitting** luôn rình rập do dữ liệu nhiều, do kiến trúc mạng quá dư thừa tham số.
- Bắt buộc áp dụng các giao thức đánh giá Hold-out hoặc K-Fold Validation đúng cách, tuyệt đối không Tuning siêu tham số trên tập Test.
- Khắc phục Overfit bằng vũ khí kỹ thuật:
 - **Thêm Dữ liệu / Data Augmentation / Feature Engineering.**
 - Thu nhỏ mạng (Reduce Network Capacity).
 - Bổ sung Phạt Trọng số (**L2 Regularization**).
 - Bổ sung Nhiều phân tách Co-adaptation (**Dropout**).
 - Ngừng sớm (**Early Stopping**).